Problem Statement

Objective:
The goal of this analysis is to help customers identify the best credit card option based on their spending habits and financial profile. We are provided with two datasets:

- **Bank Credit Card Data**: This dataset includes information about various credit card options, including cashback percentages across different categories (grocery, travel, dining), annual fees, required income, and more.
- **Customer Data**: This dataset includes information about individual customers, such as their annual income, monthly spending patterns, and preferences for cashback categories.

Using these datasets, we want to develop a system that helps customers make informed decisions about which credit card suits their spending habits, budget, and financial goals.

Use Cases

Here are 10 potential use cases that can be addressed using **Python libraries like pandas and numpy** for data analysis:

Data Prepocessing

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

customer_df=pd.read_csv("customer_data.csv")
bank_df=pd.read_csv("BankData.csv")
bank_df.head()

print(bank_df.info())
print(customer_df.info())

bank_df.columns = bank_df.columns.str.replace(" ", "_")
customer_df.columns = customer_df.columns.str.replace(" ", "_")

bank_df = bank_df.fillna(0)
customer_df = customer_df.fillna(0)
```

```
print(bank_df.head())
print(customer_df.head())

bank_df["CardType"] = bank_df["CardType"].str.lower()
bank_df["Bank_Name"] = bank_df["Bank_Name"].str.strip()
bank_df["Card_Name"] = bank_df["Card_Name"].str.strip()

bank_df = bank_df.drop_duplicates()
customer_df = customer_df.drop_duplicates()
```

1. Find the Best Credit Card for Grocery Shoppers

- **Goal**: Identify which credit card offers the highest cashback on grocery purchases (for example, from Zepto, BigBasket, etc.) based on the available percentage in the dataset.
- **Input**: Customer's monthly grocery spending.
- **Output**: The best credit card(s) based on grocery cashback rate.

```
grocery_spend=float(input("Enter your monthly grocery spending : "))

credit_cards = bank_df[bank_df["CardType"] == "credit"]

credit_cards["Cashback_Amount"] = grocery_spend *
(credit_cards["Grocery_Cashback"] / 100)

credit_cards["Discount_Amount"] = grocery_spend *
(credit_cards["Grocery_Discount"] / 100)

credit_cards["Total_Savings"] = credit_cards["Cashback_Amount"] +
credit_cards["Discount_Amount"]

best_cards = credit_cards.sort_values(by="Total_Savings", ascending=False)

print("\nBest 3 Credit Cards for Grocery Spending:\n")


for index, row in best_cards.head(3).iterrows():

    print("Card Name:", row["Card_Name"])

    print("Bank Name:", row["Bank_Name"])

    print("Cashback:", str(row["Grocery_Cashback"]) + "%")

    print("Discount:", str(row["Grocery_Discount"]) + "%")
```

```
    print("Cashback Amount: ₹", row["Cashback_Amount"])
    print("Discount Amount: ₹", row["Discount_Amount"])
    print("Total Savings: ₹", row["Total_Savings"])
    print("------------------------------------")
```

Output

```
Enter your monthly grocery spending :  45678

Best 3 Credit Cards for Grocery Spending:

Card Name: Prime Credit Card
Bank Name: Bank of Baroda
Cashback: 10.0%
Discount: 0%
Cashback Amount: ₹ 4567.8
Discount Amount: ₹ 0.0
Total Savings: ₹ 4567.8
------------------------------------
Card Name: Rupay Signet
Bank Name: Federal Bank
Cashback: 0.0%
Discount: 10%
Cashback Amount: ₹ 0.0
Discount Amount: ₹ 4567.8
Total Savings: ₹ 4567.8
------------------------------------
Card Name: IRCTC Credit Card
Bank Name: Bank of Baroda
Cashback: 10.0%
Discount: 0%
Cashback Amount: ₹ 4567.8
Discount Amount: ₹ 0.0
Total Savings: ₹ 4567.8
------------------------------------
```

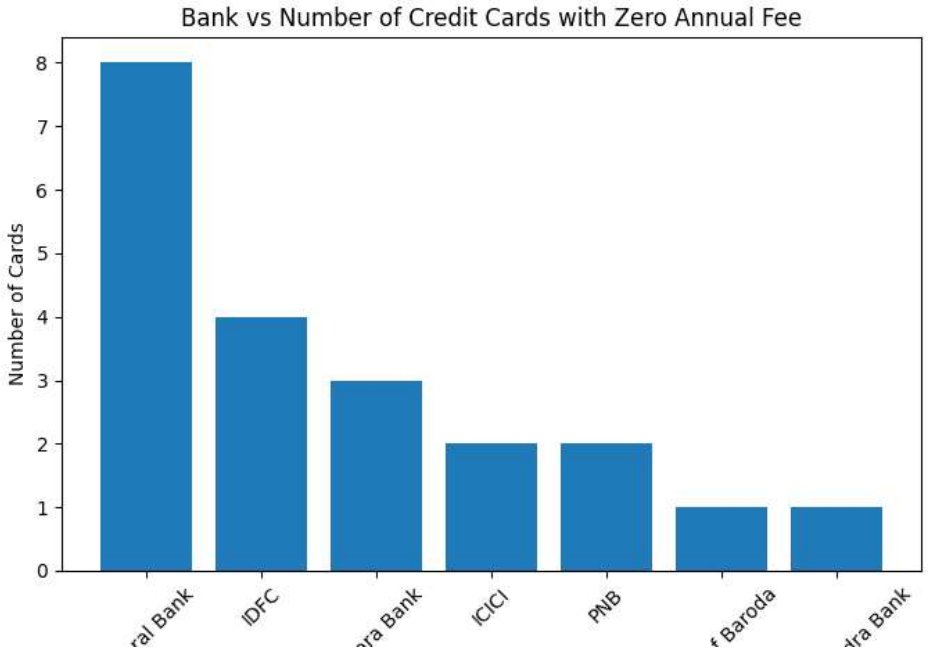
2. Identify Cards with No Annual Fee

- **Goal**: Filter out credit cards that do not have an annual fee and show them to customers who prefer no upfront costs.
- **Input**: Annual fee data.
- **Output**: List of credit cards with zero annual fee.

```
credit_cards = bank_df[bank_df["CardType"] == "credit"].copy()

no_annual_fee = credit_cards[credit_cards["Annual_Fee"] == 0]
print("\nCredit Cards with Zero Annual Fee:\n")
print(no_annual_fee[["Bank_Name", "Card_Name", "Annual_Fee"]])
bank_counts = no_annual_fee["Bank_Name"].value_counts()

plt.figure(figsize=(8,5))
plt.bar(bank_counts.index, bank_counts.values)
plt.xlabel("Bank")
plt.ylabel("Number of Cards")
plt.title("Bank vs Number of Credit Cards with Zero Annual Fee")

plt.xticks(rotation=45)

plt.show()
```

Output:

```
Credit Cards with Zero Annual Fee:

            Bank_Name                  Card_Name  Annual_Fee
0               ICICI    platinum ship credit card           0
1               ICICI        Amazon pay icici card           0
3                IDFC                   Millennia           0
4                IDFC               First Classic           0
6                IDFC                First Select           0
8                IDFC                Republic Day           0
11                 PNB               Global Classic           0
13                 PNB              Patanjali RuPay           0
28       Bank of Baroda            Prime Credit Card           0
29          Canara Bank     VISA Classic Credit Card           0
30          Canara Bank         MasterCard Standard           0
31          Canara Bank   RuPay Platinum Credit Card           0
63         Federal Bank                Visa Celesta           0
64         Federal Bank                Visa Imperio           0
65         Federal Bank                 Visa Signet           0
66         Federal Bank           Mastercard Celesta           0
67         Federal Bank           Mastercard Imperio           0
68         Federal Bank            Mastercard Signet           0
69         Federal Bank                Rupay Signet           0
70         Federal Bank                 Rupay Wave           0
77  Kotak Mahindra Bank  Kotak UPI RuPay Credit Card           0
```



3. Match Customers with Cards Based on Income

- **Goal**: Identify credit cards that are available to a customer based on their income level.
- **Input**: Customer's annual income.
- **Output**: List of credit cards the customer qualifies for based on their income.

```
bank_df["Min_Income_Required"] = bank_df["Min_Income_Required"].str.replace(",",
"").astype(int)
customer_df["total_Income"] = customer_df["total_Income"].astype(int)
customer_cards = customer_df.copy()

customer_cards["Recommended_Card"] = ""
customer_cards["Bank_Name"] = ""

for i, cust in customer_cards.iterrows():

    income = cust["total_Income"]

    eligible_cards = bank_df[bank_df["Min_Income_Required"] <= income]

    if not eligible_cards.empty:
        card = eligible_cards.iloc[0]
        customer_cards.loc[i, "Recommended_Card"] = card["Card_Name"]
```

```
        customer_cards.loc[i, "Bank_Name"] = card["Bank_Name"]

    else:
        customer_cards.loc[i, "Recommended_Card"] = "Not Eligible"
        customer_cards.loc[i, "Bank_Name"] = "N/A"

print(customer_cards[["Name","total_Income","Bank_Name","Recommended_Card"]
])
```

output:

```
                Name  total_Income Bank_Name Recommended_Card
0        Ishaan Sharma        151958       PNB  Patanjali RuPay
1            Dev Desai         46023       PNB     RuPay Select
2         Aryan Chopra        160608       PNB  Patanjali RuPay
3        Aditya Kapoor         97435       PNB     RuPay Select
4         Harsh Chopra        171699       PNB  Patanjali RuPay
...                ...           ...       ...              ...
1495      Kunal Mishra        190562       PNB  Patanjali RuPay
1496  Reyansh Malhotra        108985       PNB  Patanjali RuPay
1497      Aditya Patel        109212       PNB  Patanjali RuPay
1498      Aditya Joshi         75158       PNB     RuPay Select
1499         Sai Patel         87098       PNB     RuPay Select
```

4. Calculate Total Cashback for Each Customer

- **Goal**: Calculate the total cashback a customer would receive based on their spending across groceries, travel, and dining categories.
- **Input**: Customer's monthly spending in grocery, travel, and dining categories.
- **Output**: Total cashback value per credit card based on their spending.

```
def calculate_cashback(customer_id, grocery_spending, travel_spending,
dining_spending):

    relevant_cards =
bank_df[['Bank_Name','Card_Name','Grocery_Cashback','Travel_Cashback','Dining_
Cashback']].copy()

    # Calculate cashback for each category
    grocery_cashback = grocery_spending * (relevant_cards['Grocery_Cashback'] /
100)
    travel_cashback = travel_spending * (relevant_cards['Travel_Cashback'] / 100)
    dining_cashback = dining_spending * (relevant_cards['Dining_Cashback'] / 100)

    # Total cashback
    relevant_cards['Total_Cashback'] = grocery_cashback + travel_cashback +
```

```
dining_cashback

    return relevant_cards[['Bank_Name','Card_Name','Total_Cashback']]


# Take input from user
customer_id = int(input("Enter Customer ID: "))
grocery_spending = float(input("Enter monthly grocery spending: "))
travel_spending = float(input("Enter monthly travel spending: "))
dining_spending = float(input("Enter monthly dining spending: "))

# Call function
cashback_info = calculate_cashback(customer_id, grocery_spending,
travel_spending, dining_spending)

print("\nTotal Cashback for Each Credit Card:\n")
print(cashback_info)
```

Output:

```
Enter Customer ID:  1
Enter monthly grocery spending:  345
Enter monthly travel spending:  456
Enter monthly dining spending:  6789

Total Cashback for Each Credit Card:

            Bank_Name                  Card_Name  Total_Cashback
0               ICICI    platinum ship credit card        1032.645
1               ICICI        Amazon pay icici card         236.820
2               ICICI            Coral Credit Card        1420.650
3                IDFC                   Millennia           3.450
4                IDFC               First Classic           0.000
..                ...                        ...             ...
73  Kotak Mahindra Bank  IndianOil Kotak Credit Card           6.900
74  Kotak Mahindra Bank    Zen Signature Credit Card           0.000
75  Kotak Mahindra Bank        Kotak Air Credit Card          13.680
76  Kotak Mahindra Bank   PVR INOX Kotak Credit Card         339.450
77  Kotak Mahindra Bank  Kotak UPI RuPay Credit Card           0.000
```

5. Identify Cards with Best Dining Cashback

- **Goal**: Find the credit cards that offer the highest cashback on dining (e.g., through Swiggy, Zomato).
- **Input**: Customer's monthly dining spending.
- **Output**: List of cards with the highest dining cashback percentage.

```
results = []

for _, cust in customer_df.iterrows():

    name = cust["Name"]
    dining_spend = cust["Dining"]

    for _, card in bank_df.iterrows():

        dining_cb = dining_spend * (card["Dining_Cashback"] / 100)

        results.append({
            "Customer_Name": name,
            "Bank_Name": card["Bank_Name"],
            "Card_Name": card["Card_Name"],
            "Dining_Cashback_Amount": round(dining_cb,2)
        })

dining_df = pd.DataFrame(results)

# Best card per customer
best_cards = dining_df.sort_values("Dining_Cashback_Amount", ascending=False)

print(best_cards.head(10))
```

Output:

```
       Customer_Name  Bank_Name                    Card_Name  \
46297     Dhruv Joshi  Axis Bank  Axis Bank Delight Debit Card
105733   Atharva Rao  Axis Bank  Axis Bank Delight Debit Card
80929     Kabir Nair  Axis Bank  Axis Bank Delight Debit Card
14083   Aditya Mehta  Axis Bank  Axis Bank Delight Debit Card
44035   Ishaan Gupta  Axis Bank  Axis Bank Delight Debit Card
93955   Ishaan Gupta  Axis Bank  Axis Bank Delight Debit Card
21259     Aarav Iyer  Axis Bank  Axis Bank Delight Debit Card
24067    Kunal Gupta  Axis Bank  Axis Bank Delight Debit Card
74065    Aryan Verma  Axis Bank  Axis Bank Delight Debit Card
62521    Harsh Desai  Axis Bank  Axis Bank Delight Debit Card

       Dining_Cashback_Amount
46297                 3749.00
105733                 3748.00
80929                 3747.00
14083                 3741.00
44035                 3740.00
93955                 3739.50
21259                 3738.25
```

24067	3736.75
74065	3736.00
62521	3730.50

6. Card Comparison: Annual Fee vs. Cashback

- **Goal**: Compare the annual fee of credit cards with the total cashback value a customer would get.
- **Input**: Customer's total annual spending in different categories.
- **Output**: List of cards with the best value when considering both the annual fee and cashback.

```
results = []

for _, cust in customer_df.iterrows():

    name = cust["Name"]
    grocery = cust["Grocery"]
    travel = cust["Travelling"]
    dining = cust["Dining"]

    for _, card in bank_df.iterrows():

        grocery_cb = grocery * (card["Grocery_Cashback"] / 100)
        travel_cb = travel * (card["Travel_Cashback"] / 100)
        dining_cb = dining * (card["Dining_Cashback"] / 100)

        total_cashback = grocery_cb + travel_cb + dining_cb
        net_value = total_cashback - card["Annual_Fee"]

        results.append({
            "Customer_Name": name,
            "Bank_Name": card["Bank_Name"],
            "Card_Name": card["Card_Name"],
            "Annual_Fee": card["Annual_Fee"],
            "Total_Cashback": round(total_cashback,2),
            "Net_Value": round(net_value,2)
        })

comparison_df = pd.DataFrame(results)

# Show best value cards
best_cards = comparison_df.sort_values(by="Net_Value", ascending=False)

print("\nTop Credit Cards Based on Value:\n")
```

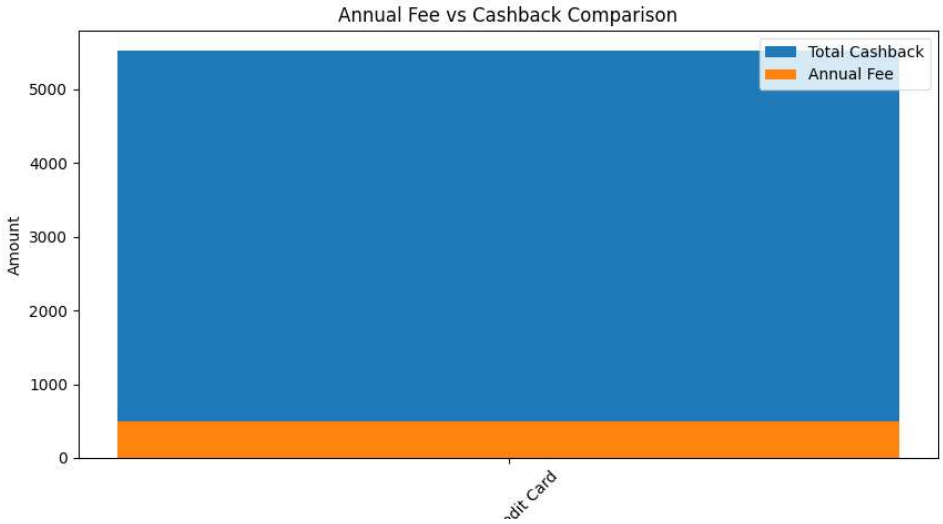
```
print(best_cards.head(10)
[["Customer_Name","Bank_Name","Card_Name","Annual_Fee","Total_Cashback","
Net_Value"]])

# Visualization
top_cards = best_cards.head(10)

plt.figure(figsize=(10,5))

plt.bar(top_cards["Card_Name"], top_cards["Total_Cashback"], label="Total
Cashback")
plt.bar(top_cards["Card_Name"], top_cards["Annual_Fee"], label="Annual Fee")

plt.xlabel("Credit Card")
plt.ylabel("Amount")
plt.title("Annual Fee vs Cashback Comparison")

plt.xticks(rotation=45)

plt.legend()

plt.show()
```

Output:

Top Credit Cards Based on Value:

```
        Customer_Name Bank_Name          Card_Name  Annual_Fee  \
105692     Atharva Rao     ICICI  Coral Credit Card         500
54134      Ayaan Gupta     ICICI  Coral Credit Card         500
73868         Sai Iyer     ICICI  Coral Credit Card         500
43994     Ishaan Gupta     ICICI  Coral Credit Card         500
26756   Reyansh Sharma     ICICI  Coral Credit Card         500
53276   Reyansh Bansal     ICICI  Coral Credit Card         500
2108     Ishaan Pillai     ICICI  Coral Credit Card         500
47114    Aditya Sharma     ICICI  Coral Credit Card         500
8582      Ishaan Gupta     ICICI  Coral Credit Card         500
26444      Rohan Verma     ICICI  Coral Credit Card         500

        Total_Cashback  Net_Value
105692         5518.30    5018.30
54134          5451.70    4951.70
73868          5416.85    4916.85
43994          5360.25    4860.25
26756          5328.95    4828.95
53276          5293.25    4793.25
2108           5292.35    4792.35
47114          5289.75    4789.75
```

8582 5272.00 4772.00
26444 5259.95 4759.95



7. Filter Cards Based on Monthly Spending Limit

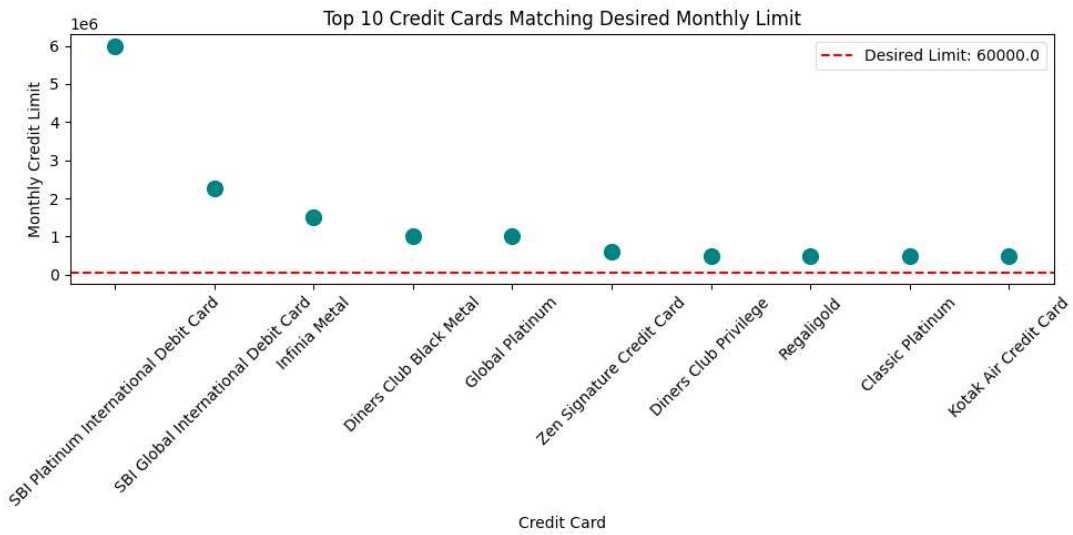
- **Goal**: Identify cards that offer a monthly spending limit suitable for the customer's financial needs.
- **Input**: Customer's desired monthly credit limit.
- **Output**: List of credit cards with monthly spending limits that match the customer's preferences.

```
desired_limit = float(input("Enter your desired monthly credit limit: "))

# Clean the 'Limit' column
bank_df["Limit"] = bank_df["Limit"].astype(str)   # ensure string
bank_df["Limit"] = bank_df["Limit"].str.replace(",", "")  # remove commas
bank_df["Limit"] = pd.to_numeric(bank_df["Limit"], errors="coerce")  # convert to float

# Drop rows where conversion failed
bank_df = bank_df.dropna(subset=["Limit"])

# Filter cards that satisfy the desired limit
filtered_cards = bank_df[bank_df["Limit"] >= desired_limit]

# Sort descending by limit and take top 10
top_cards = filtered_cards.sort_values(by="Limit", ascending=False).head(10)

print("\nTop 10 Credit Cards Matching Your Monthly Limit:\n")
print(top_cards[["Bank_Name","Card_Name","Limit"]])
```

```
# Scatter Plot
plt.figure(figsize=(10,5))
plt.scatter(top_cards["Card_Name"], top_cards["Limit"], color='teal', s=100)

# Horizontal line for customer requirement
plt.axhline(y=desired_limit, color='red', linestyle='--', label=f"Desired Limit:
{desired_limit}")

plt.xlabel("Credit Card")
plt.ylabel("Monthly Credit Limit")
plt.title("Top 10 Credit Cards Matching Desired Monthly Limit")
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()
```

Output:

```
Enter your desired monthly credit limit:  50000

Top 10 Credit Cards Matching Your Monthly Limit:

           Bank_Name                          Card_Name    Limit
25               SBI  SBI Platinum International Debit Card  6000000
24               SBI    SBI Global International Debit Card  2250000
54         HDFC Bank                        Infinia Metal  1500000
53         HDFC Bank                Diners Club Black Metal  1000000
9               PNB                      Global Platinum  1000000
74  Kotak Mahindra Bank             Zen Signature Credit Card   600000
75  Kotak Mahindra Bank                Kotak Air Credit Card   500000
52         HDFC Bank                Diners Club Privilege   500000
51         HDFC Bank                          Regaligold   500000
57         HDFC Bank                  Preferred Platinum   500000
```



8. Find Cards with Highest Rewards Points

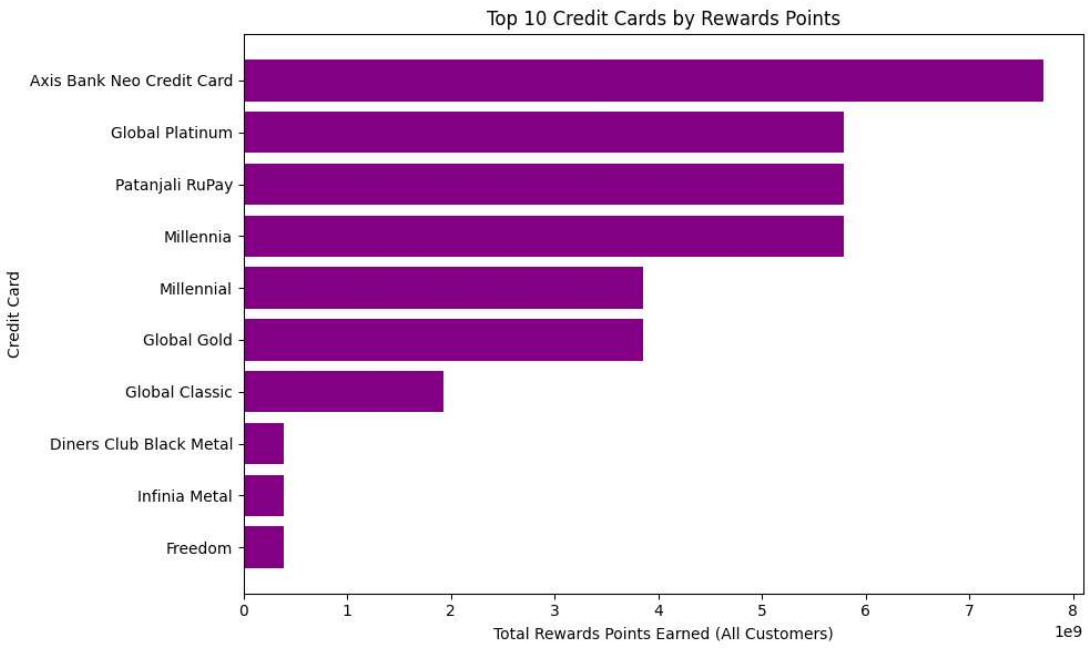
- **Goal**: Find credit cards that provide the highest reward points for every dollar spent.
- **Input**: Customer's monthly expenditure across different categories.
- **Output**: List of credit cards offering the highest reward points.

```
results = []

# Loop through each customer
for _, cust in customer_df.iterrows():
    grocery = cust["Grocery"]
    travel = cust["Travelling"]
    dining = cust["Dining"]
    total_spending = grocery + travel + dining

    # Loop through each credit card
    for _, card in bank_df.iterrows():
        reward_points_earned = total_spending * card["Rewards_Points"]
        results.append({
            "Bank_Name": card["Bank_Name"],
            "Card_Name": card["Card_Name"],
            "Rewards_Points": reward_points_earned
        })

# Convert results to DataFrame
rewards_df = pd.DataFrame(results)
```

```
# Aggregate by Card_Name + Bank_Name to sum rewards points across all
customers
agg_rewards = rewards_df.groupby(["Bank_Name","Card_Name"])
["Rewards_Points"].sum().reset_index()

# Get top 10 cards by total rewards points
top_rewards = agg_rewards.sort_values(by="Rewards_Points",
ascending=False).head(10)

print("\nTop 10 Credit Cards (Aggregated Rewards Points Across All Customers):\n")
print(top_rewards)

# --- Horizontal Bar Chart ---
plt.figure(figsize=(10,6))
plt.barh(top_rewards["Card_Name"], top_rewards["Rewards_Points"], color='purple')
plt.xlabel("Total Rewards Points Earned (All Customers)")
plt.ylabel("Credit Card")
plt.title("Top 10 Credit Cards by Rewards Points")
plt.gca().invert_yaxis()  # highest on top
plt.tight_layout()
plt.show()
```

Output:

```
Top 10 Credit Cards (Aggregated Rewards Points Across All Customers):

   Bank_Name                Card_Name  Rewards_Points
5   Axis Bank  Axis Bank Neo Credit Card    7.715612e+09
66        PNB            Global Platinum    5.786709e+09
68        PNB            Patanjali RuPay    5.786709e+09
54       IDFC                 Millennia    5.786709e+09
67        PNB                Millennial    3.857806e+09
65        PNB               Global Gold    3.857806e+09
64        PNB             Global Classic    1.928903e+09
30  HDFC Bank    Diners Club Black Metal    3.857806e+08
34  HDFC Bank              Infinia Metal    3.857806e+08
32  HDFC Bank                    Freedom    3.857806e+08
```



9. Lounge Access Preferences

- **Goal**: Identify credit cards that offer lounge access and match the customer's preferences for travel benefits.
- **Input**: Customer’s travel habits and lounge access requirements.
- **Output**: List of credit cards with lounge access, categorized by frequency per year.

```
#case 9

import pandas as pd

import matplotlib.pyplot as plt


# Add default customer lounge preference if not present

if 'Preferred_Lounge_Access' not in customer_df.columns:

    customer_df['Preferred_Lounge_Access'] = 1  # default minimum


# Convert 'Lounge_access_per year' to numeric

bank_df["Lounge_access_per_year"] = pd.to_numeric(bank_df["Lounge_access_per
```

```
year"], errors="coerce")

bank_df = bank_df.dropna(subset=["Lounge_access_per_year"])


# Determine minimum lounge access requirement across all customers

min_lounges_required = customer_df["Preferred_Lounge_Access"].max()  # ensures
all customers satisfied


# Filter cards meeting the requirement

lounge_cards = bank_df[bank_df["Lounge_access_per_year"] >=
min_lounges_required]


print(f"\nCredit Cards Offering >= {min_lounges_required} Lounge Access Per Year
(All Customers):\n")

print(lounge_cards[["Bank_Name","Card_Name","Lounge_access_per_year"]])


# Visualization – Stacked Bar Chart by Bank

bank_lounge_counts =
lounge_cards.groupby(["Bank_Name","Lounge_access_per_year"]).size().unstack(fill
_value=0)


bank_lounge_counts.plot(kind="bar", stacked=True, figsize=(10,6),
colormap="viridis")

plt.xlabel("Bank Name")

plt.ylabel("Number of Credit Cards")

plt.title(f"Credit Cards with Lounge Access >= {min_lounges_required} (All
Customers)")

plt.xticks(rotation=45)

plt.legend(title="Lounge Access / Year")

plt.tight_layout()

plt.show()
```

Output:

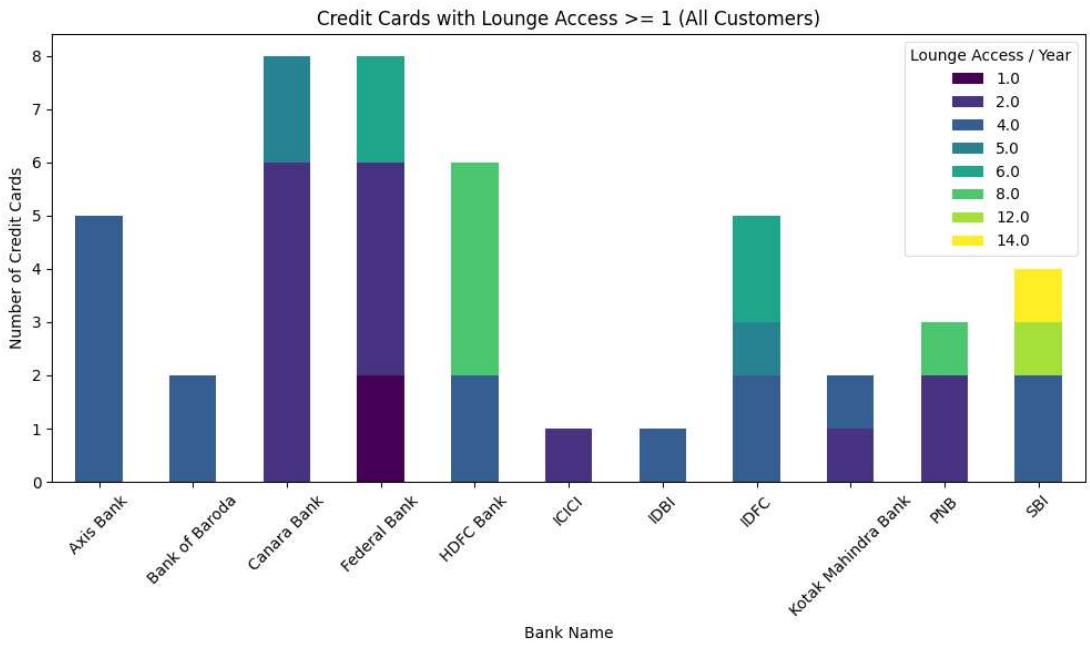
Credit Cards Offering >= 1 Lounge Access Per Year (All Customers):

	Bank_Name	Card_Name \
2	ICICI	Coral Credit Card
3	IDFC	Millennia
4	IDFC	First Classic
5	IDFC	WOW ! Black
6	IDFC	First Select
8	IDFC	Republic Day
9	PNB	Global Platinum
10	PNB	Global Gold
12	PNB	Millennial
16	IDBI	Royale Signature card
18	SBI	Cashback SBI Card
21	SBI	SBI Card Prime
22	SBI	SBI Card ELITE
25	SBI	SBI Platinum International Debit Card
26	Bank of Baroda	HPCL Credit Card
27	Bank of Baroda	IRCTC Credit Card
29	Canara Bank	VISA Classic Credit Card
30	Canara Bank	MasterCard Standard
31	Canara Bank	RuPay Platinum Credit Card
32	Canara Bank	Canara Gold Credit Card
33	Canara Bank	RuPay Debit Card
34	Canara Bank	VISA Debit Card
35	Canara Bank	MasterCard Debit Card
36	Canara Bank	EMV Chip Debit Card
37	Axis Bank	Axis Bank ACE Credit Card
40	Axis Bank	Airtel Axis Bank Credit Card
41	Axis Bank	Flipkart Axis Bank Credit Card
42	Axis Bank	Axis Bank MyZone Credit Card
43	Axis Bank	Axis Bank Delight Debit Card
55	HDFC Bank	Classic Platinum
56	HDFC Bank	Platinum Debit
57	HDFC Bank	Preferred Platinum
58	HDFC Bank	Millennia Debit
61	HDFC Bank	Vishesh Debit
62	HDFC Bank	Giga Business
63	Federal Bank	Visa Celesta
64	Federal Bank	Visa Imperio
65	Federal Bank	Visa Signet
66	Federal Bank	Mastercard Celesta
67	Federal Bank	Mastercard Imperio
68	Federal Bank	Mastercard Signet
69	Federal Bank	Rupay Signet

70	Federal Bank	Rupay Wave
74	Kotak Mahindra Bank	Zen Signature Credit Card
75	Kotak Mahindra Bank	Kotak Air Credit Card

	Lounge_access_per_year
2	2.0
3	5.0
4	6.0
5	4.0
6	6.0
8	4.0
9	8.0
10	2.0
12	2.0
16	4.0
18	4.0
21	12.0
22	14.0
25	4.0
26	4.0
27	4.0
29	2.0
30	2.0
31	5.0
32	5.0
33	2.0
34	2.0
35	2.0
36	2.0
37	4.0
40	4.0
41	4.0
42	4.0
43	4.0
55	8.0
56	8.0
57	8.0
58	4.0
61	8.0
62	4.0
63	2.0
64	6.0
65	2.0
66	2.0
67	6.0
68	2.0
69	1.0

70	1.0
74	2.0
75	4.0



10. Customer-Specific Credit Card Recommendations

- **Goal**: Provide personalized credit card recommendations based on a customer's profile, which includes income, spending habits (grocery, dining, travel), and reward preferences.
- **Input**: Customer's annual income, monthly spending in different categories (grocery, travel, dining), and rewards/benefits preference.
- **Output**: Recommended credit card(s) that align with the customer's needs.

```
import pandas as pd

import matplotlib.pyplot as plt


# Convert numeric columns in bank_df

bank_df["Min_Income_Required"] =
pd.to_numeric(bank_df["Min_Income_Required"], errors="coerce")

bank_df["Grocery_Cashback"] = pd.to_numeric(bank_df["Grocery_Cashback"],
```

```
errors="coerce")

bank_df["Travel_Cashback"] = pd.to_numeric(bank_df["Travel_Cashback"],
errors="coerce")

bank_df["Dining_Cashback"] = pd.to_numeric(bank_df["Dining_Cashback"],
errors="coerce")

bank_df["Rewards_Points"] = pd.to_numeric(bank_df["Rewards_Points"],
errors="coerce")

bank_df["Annual_Fee"] = pd.to_numeric(bank_df["Annual_Fee"], errors="coerce")

bank_df["Lounge_access_per_year"] = pd.to_numeric(bank_df["Lounge_access_per
year"], errors="coerce")


# Convert customer income

customer_df["total_Income"] = pd.to_numeric(customer_df["total_Income"],
errors="coerce")


# Store top card recommendations

top_recommendations = []


# Loop through first 5 customers

for _, cust in customer_df.head(5).iterrows():

    name = cust["Name"]

    income = cust["total_Income"]

    grocery = cust["Grocery"]

    travel = cust["Travelling"]

    dining = cust["Dining"]

    total_spending = grocery + travel + dining


    # Optional lounge preference

    preferred_lounges = 2 if travel > 4000 else 0
```

```
    # Filter eligible cards
    eligible_cards = bank_df[
        (bank_df["Min_Income_Required"] <= income) &
        (bank_df["Lounge_access_per_year"] >= preferred_lounges)
    ]

    if eligible_cards.empty:
        print(f"\nCustomer {name} has no eligible cards.\n")
        continue

    # Calculate total value = cashback + rewards - annual fee
    eligible_cards = eligible_cards.copy()
    eligible_cards["Cashback_Amount"] = (grocery *
eligible_cards["Grocery_Cashback"] / 100) + \
                                (travel * eligible_cards["Travel_Cashback"] / 100) + \
                                (dining * eligible_cards["Dining_Cashback"] / 100)
    eligible_cards["Rewards_Amount"] = total_spending *
eligible_cards["Rewards_Points"]
    eligible_cards["Total_Value"] = eligible_cards["Cashback_Amount"] +
eligible_cards["Rewards_Amount"] - eligible_cards["Annual_Fee"]

    # Pick the single best card
    best_card = eligible_cards.sort_values(by="Total_Value", ascending=False).iloc[0]

    top_recommendations.append({
        "Customer": name,
        "Bank": best_card["Bank_Name"],
        "Card": best_card["Card_Name"],
        "Total_Value": round(best_card["Total_Value"],2),
```

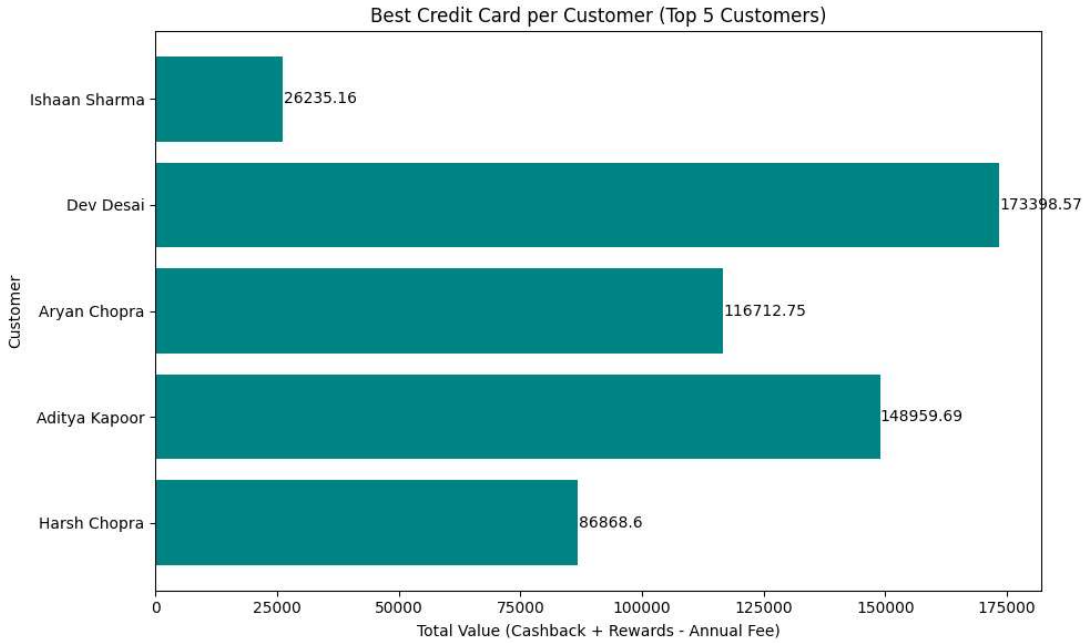
```
        "Cashback": round(best_card["Cashback_Amount"],2),
        "Rewards": round(best_card["Rewards_Amount"],2),
        "Annual_Fee": best_card["Annual_Fee"],
        "Lounge_Access": best_card["Lounge_access_per_year"]
    })

# Convert to DataFrame
top_cards_df = pd.DataFrame(top_recommendations)

# Show the results
print("\nBest Credit Card for Each of the First 5 Customers:\n")
print(top_cards_df)

# --- Visualization ---
plt.figure(figsize=(10,6))
plt.barh(top_cards_df["Customer"], top_cards_df["Total_Value"], color='teal')
for i, v in enumerate(top_cards_df["Total_Value"]):
    plt.text(v + 50, i, str(v), va='center')  # show value on bars

plt.xlabel("Total Value (Cashback + Rewards - Annual Fee)")
plt.ylabel("Customer")
plt.title("Best Credit Card per Customer (Top 5 Customers)")
plt.gca().invert_yaxis()  # highest value on top
plt.tight_layout()
plt.show()
```

Output:

Best Credit Card for Each of the First 5 Customers:

	Customer	Bank	Card	Total_Value	Cashback \
0	Ishaan Sharma	Canara Bank	MasterCard Standard	26235.16	87.16
1	Dev Desai	HDFC Bank	Freedom	173398.57	718.57
2	Aryan Chopra	Canara Bank	MasterCard Standard	116712.75	387.75
3	Aditya Kapoor	HDFC Bank	Freedom	148959.69	549.69
4	Harsh Chopra	Canara Bank	MasterCard Standard	86868.60	288.60

	Rewards	Annual_Fee	Lounge_Access
0	26148.0	0	2.0
1	173180.0	500	0.0
2	116325.0	0	2.0
3	148910.0	500	0.0
4	86580.0	0	2.0



Approach

We can use **pandas** and **numpy** to perform the following tasks:

- **Data Cleaning & Preprocessing**: Handle missing values, outliers, and data type corrections.
- **Data Merging**: Join the credit card dataset with the customer dataset to evaluate customer preferences.
- **Data Filtering**: Use pandas filtering methods to select rows based on specific conditions (e.g., finding cards with no annual fee).
- **Calculations**: Calculate cashback amounts, rewards points, and other metrics based on customer spending.
- **Visualization**: Use **matplotlib** or **seaborn** to create visualizations (e.g., bar charts for cashback, pie charts for card type distributions).